

```

# This dict maps each card rank to a value for a standard deck
STANDARD_DICT = {'Ace':1, '2':2, '3':3, '4':4, '5':5,
                  '6':6, '7':7, '8':8, '9':9, '10':10,
                  'Jack':11, 'Queen':12, 'King':13}

❷ def __init__(self, window, rankValueDict=STANDARD_DICT):
    # rankValueDict defaults to STANDARD_DICT, but you can call it
    # with a different dict, e.g., a special dict for Blackjack
    self.startingDeckList = []
    self.playingDeckList = []
    for suit in Deck.SUIT_TUPLE:
        ❸ for rank, value in rankValueDict.items():
            oCard = Card(window, rank, suit, value)
            self.startingDeckList.append(oCard)

    self.shuffle()

❹ def shuffle(self):
    # Copy the starting deck and save it in the playing deck list
    self.playingDeckList = self.startingDeckList.copy()
    for oCard in self.playingDeckList:
        oCard.conceal()
    random.shuffle(self.playingDeckList)

❺ def getCard(self):
    if len(self.playingDeckList) == 0:
        raise IndexError('No more cards')
    # Pop one card off the deck and return it
    oCard = self.playingDeckList.pop()
    return oCard

❻ def returnCardToDeck(self, oCard):
    # Put a card back into the deck
    self.deckList.insert(0, oCard)

```

Listing 12-2: A `Deck` class that manages 52 card objects

We begin the `Deck` class by creating a few class variables ❶ that we'll use to create 52 cards with the proper suits and values. There are only four methods.

To the `__init__()` method ❷, we pass a reference to the window and an optional dictionary that maps card ranks to their values. If none is passed in,